

C++

BUBBLE SORT

Lecture-21

Raghav Garg

COLLEGE
WALLAH

Today's checklist

- 1) **Sorting**
- 2) **Bubble sort Algorithm**
- 3) **Time complexity and space complexity**
- 4) **Bubble sort optimization**
- 5) **Stable and unstable sort**
- 6) 2 Questions

What is sorting?

arr =

5	3	1	4	2
---	---	---	---	---

To Sort = put in ascending order

Increasing order ↗

Non-Decreasing ↗

↘ 1 2 3 4 5

Sort in
decreasing
order

→ put the elements in
descending order

→ 5 4 3 2 1

Bubble sort algorithm

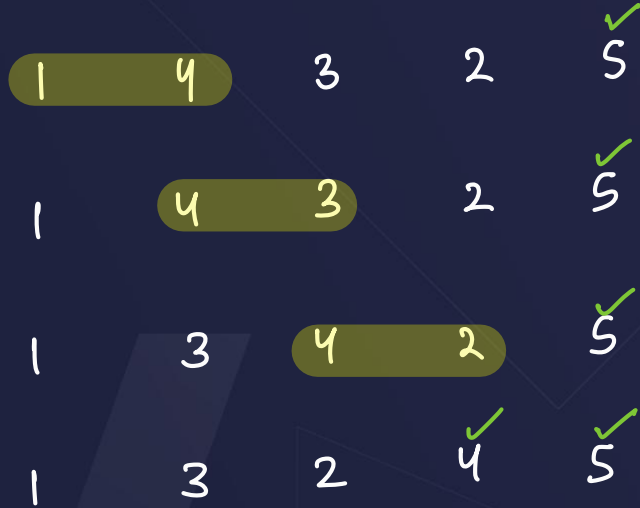
1st Pass

arr =

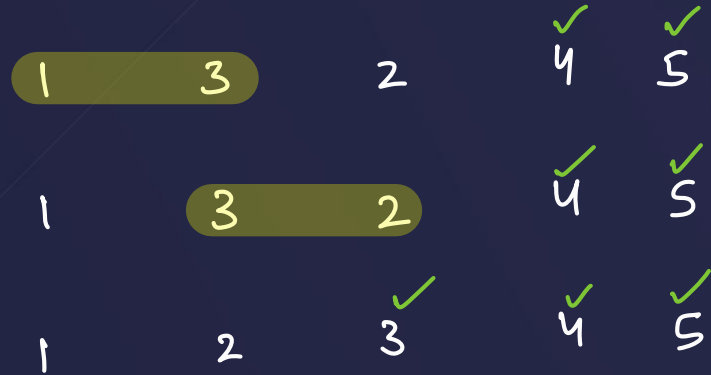
	5	1	4	3	2
1	5	4	3	2	
1	4	5	3	2	
1	4	3	5	2	
1	4	3	2	5	✓

Bubble sort algorithm

2nd Pass



3rd Pass

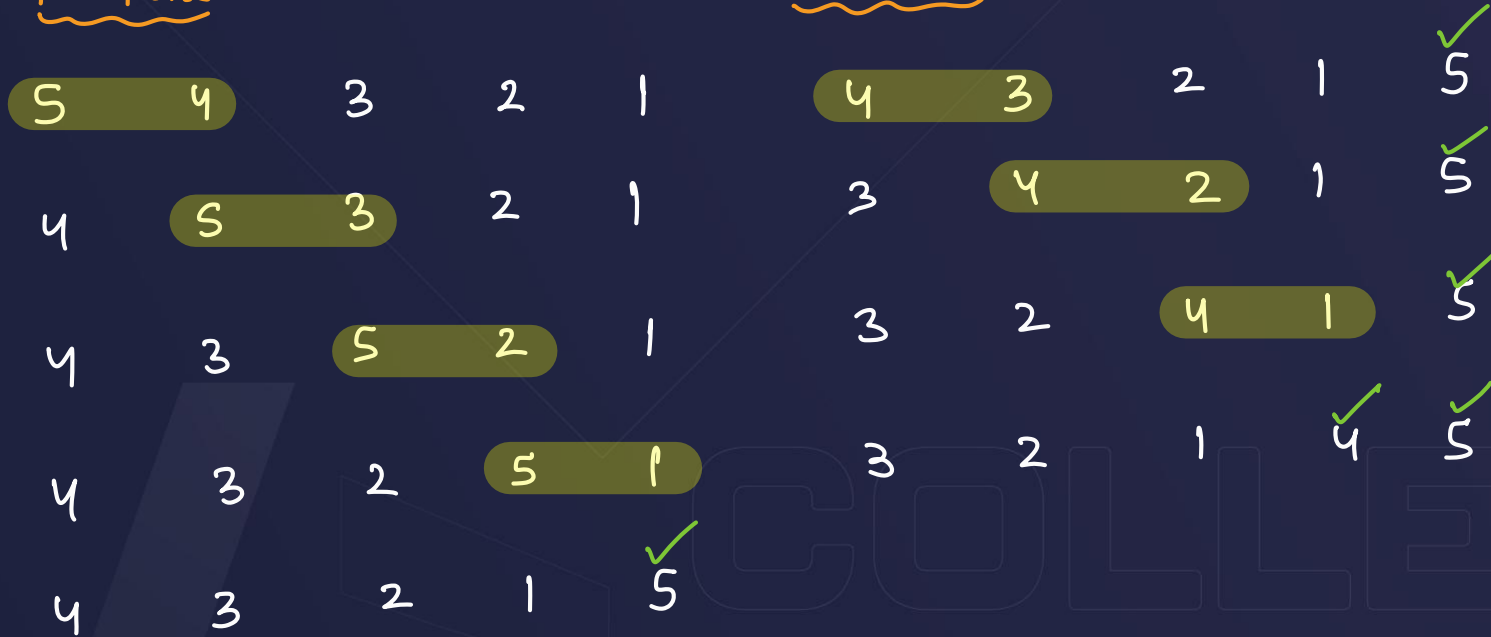


COLLEGE
WALLAH

Bubble sort algorithm

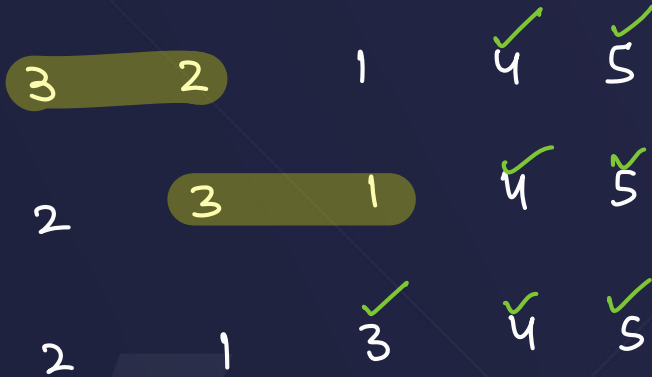
1st Pass :

2nd Pass:



Bubble sort algorithm

3rd Pass :



4th pass :



Formula $\rightarrow \frac{n(n-1)}{2} \rightarrow \frac{5(4)}{2} = 10$ ✓

Observations :

- In each pass the n^{th} max element goes to its right position
- If there are 'n' elements, then we require almost 'n-1' passes to sort.

Algorithm : In each pass swap 2 adjacent elements if $\text{arr}[i] > \text{arr}[i+1]$
Iteration in each pass also reduces.

```
// bubble sort
for(int i=0;i<n-1;i++){ // n-1 passes
    // traverse
    for(int j=0;j<n-1;j++){
        if(arr[j]>arr[j+1]){ //swap
            swap(arr[j],arr[j+1]);
        }
    }
}
```

T.C. $\rightarrow O(n^2)$

arr =

0	1	2	3
4	3	2	1
3	4	2	1
3	2	4	1
3	2	1	4 ✓

$n=4$

2nd pass

3	2	1	4
2	3	1	4
2	1	3	4
2	1	3	4

3rd pass

2	1	3	4
1	2	3	4
1	2	3	4
1	2	3	4

T.n.O $\rightarrow (n-1) \times (n-1)$

Time and Space complexity

```
// bubble sort
for(int i=0;i<n-1;i++){ // n-1 passes
    // traverse
    for(int j=0;j<n-1-i;j++){
        if(arr[j]>arr[j+1]){ //swap
            swap(arr[j],arr[j+1]);
        }
    }
}
```

$i = 0, 1, 2, \dots, n-2$

$i=0, j = 0, 1, 2, \dots, n-2$ $n-1$

$i=1, j = 0, 1, 2, \dots, n-3$ $n-2$

$i=2, j = 0, 1, 2, \dots, n-4$ $n-3$

$\vdots$

$i=n-2, j = 0$ $\rightarrow 2$
 $\rightarrow 1$

$$T.C. \rightarrow 1 + 2 + 3 + \dots + n-1$$

$$= \frac{(n-1)(n-1+1)}{2} = \frac{n(n-1)}{2} \rightarrow T.C. \rightarrow O(n^2)$$

Time and Space complexity

Time Complexity $\rightarrow O(n^2)$

Space Complexity $\rightarrow O(1)$

COLLEGE
WALLAH

Can we optimize it further?

1st Pass

2nd pass

5	1	2	3	4	1	2	3	4	5
1	5	2	3	4	1	2	3	4	5
1	2	5	3	4	1	2	3	4	5
1	2	3	5	4	1	2	3	4	5
1	2	3	4	5	1	2	3	4	5
1	2	3	4	5	1	2	3	4	5

↓
Sorted

break ;

Can we optimize it further ?

```
// bubble sort optimised
for(int i=0;i<n-1;i++){ // n-1 passes
    // traverse
    bool flag = true;  TRUE MEANS SORTED
    for(int j=0;j<n-1-i;j++){
        if(arr[j]>arr[j+1]){ //swap
            swap(arr[j],arr[j+1]);
            flag = false;
        }
    }
    if(flag==true){ // swap didn't happen
        break;
    }
}
```

Time Complexity :

Best Case : $O(n)$

Avg. Case : $O(n^2)$

Worst Case : $O(n^2)$

Ques: Given an array, find if it is sorted or not

→ arr = { 1, 2, 3, 4 } $O(n)$

```
bool flag = true; //sorted
for (int i=0; i<n-1; i++){
    if (arr[i] > arr[i+1]){
        flag = false;
        break;
    }
}
```

if (flag == true) → sorted
else → unsorted

Stable and Unstable sort

arr $\rightarrow$ 1 2 3 S_1 S_2

Conclusion: Bubble Sort is a stable sort

after sort $\rightarrow$ 1 2 3 S_1 $S_2 \rightarrow$ Stable

$\rightarrow$ 1 2 3 S_2 $S_1 \rightarrow$ Unstable

Ques : How much maximum swaps are needed to sort array of length 6 ?

→ arr = 6 5 4 3 2 1

↓

total no. of swap = total no. of operations

$$= \frac{n(n-1)}{2} = \frac{6 \cdot 5}{2} = 15 \text{ swaps}$$

Ques : Sort a String in decreasing order of values associated after removal of values smaller than X.

String s = "AZ4ZXB DJKX";

str = "Z4ZX X";

sort ↪ = ZZ4XX

COLLEGE
WALLAH

Ques : Push zeroes to end while maintaining the relative order of other elements.

'Bubble Sort ki Importance'

$arr = \{ 5, 0, 1, 2, 0, 0, 4, 0, 3 \}$

$arr = \{ 5, 1, 2, 4, 3, 0, 0, 0, 0 \}$

Summary : Bubble Sort $\rightarrow$ T.C. $\rightarrow O(n^2)$
Inbuilt Sort $\rightarrow$ T.C. $\rightarrow O(n \cdot \log n)$

THANK YOU

COLLEGE
WALLAH